

Construção de um Compilador para Pascal Standard

Processamento de Linguagens 2025/2026

Nome: João Veloso Número de Aluno: a104434

1. Introdução

Este relatório descreve a implementação de um compilador para Pascal Standard, desenvolvido no âmbito da unidade curricular de Processamento de Linguagens. O compilador processa código-fonte Pascal e traduz diretamente para instruções da máquina virtual EWVM disponibilizada pelos docentes, sem passar por uma representação intermédia própria, optei por esta opção descrita no enunciado do projeto.

O objetivo, para além de fazer o compilador "funcionar" nos exemplos fornecidos, foi construir um pipeline com fases bem separadas e responsabilidades claras, de forma a que cada etapa (léxica, sintática, semântica, geração de código) possa ser entendida, testada e discutida de forma independente das restantes.

O projeto está organizado em quatro fases clássicas de um compilador, cada uma isolada no seu próprio módulo Python, orquestradas sequencialmente por `main.py`. Cada fase só é executada se a anterior tiver terminado sem erros: o pipeline aborta assim que um erro é detetado, com uma mensagem dirigida ao utilizador e o número da linha onde ocorreu sempre que aplicável.

2. Arquitetura Geral

O pipeline de compilação segue o seguinte fluxo:



Cada módulo tem uma responsabilidade única:

Módulo	Responsabilidade
<code>lexer.py</code>	Converte texto em tokens
<code>parser.py</code>	Constrói a AST a partir dos tokens
<code>ast_nodes.py</code>	Define os nós da AST (tipos, declarações, instruções, expressões)
<code>semantic.py</code> / <code>semantic_errors.py</code>	Valida tipos, declarações e âmbitos
<code>codegen.py</code>	Traduz a AST validada em instruções EWVM
<code>main.py</code>	Orquestra as quatro fases e escreve o <code>.vm</code> de saída

Esta separação em módulos independentes tem duas vantagens práticas:

- cada fase pode ser testada e discutida isoladamente (por exemplo, correr só o parser sobre um excerto de código e inspecionar a AST resultante, sem gerar código nenhum);
- torna explícito que cada fase tem uma única responsabilidade, por exemplo, o parser não sabe nada sobre tipos, o semantic não sabe nada sobre instruções da EWVM, e o codegen não valida nada, confia inteiramente na AST validada que recebe.

`main.py` reflete esta filosofia diretamente na função `compilar`, que corre as quatro etapas pela ordem do enunciado e devolve as instruções finais, sendo depois escritas num ficheiro `.vm` com a mesma base do nome do ficheiro de entrada (ou um caminho explícito, via `-o`).

3. Análise Léxica

A análise léxica está implementada em `lexer.py`, usando a ferramenta `ply.lex`, tal como pedido no enunciado.

3.1 Tokens reconhecidos

O lexer reconhece dois grupos de tokens: os símbolos e operadores (definidos por expressões regulares simples, como `t_ASSIGN = r':='`); e as palavras reservadas do Pascal, definidas num único dicionário (ordenado alfabeticamente):

```
reserved = {
    'and': 'AND', 'array': 'ARRAY', 'begin': 'BEGIN', 'boolean': 'BOOLEAN_TYPE', 'div': 'DIV', 'downto':
    'DOWNTO', 'do': 'DO', 'else': 'ELSE', 'end': 'END', 'false': 'FALSE', 'for': 'FOR', 'function':
    'FUNCTION', 'if': 'IF', 'integer': 'INTEGER_TYPE', 'mod': 'MOD', 'not': 'NOT', 'of': 'OF', 'or': 'OR',
    'procedure': 'PROCEDURE', 'program': 'PROGRAM', 'readln': 'READLN', 'string': 'STRING_TYPE', 'then':
    'THEN', 'to': 'TO', 'true': 'TRUE', 'var': 'VAR', 'while': 'WHILE', 'writeln': 'WRITELN',
}
```

Em vez de se escrever uma regra separada para cada palavra-chave, existe uma única regra `t_ID` que reconhece qualquer identificador válido e depois consulta este dicionário para decidir se se trata de uma palavra reservada ou de um identificador comum:

```
def t_ID(t):
    r"[a-zA-Z_][a-zA-Z0-9_]*"
    t.type = reserved.get(t.value.lower(), 'ID')
    return t
```

A comparação é feita em minúsculas (`t.value.lower()`), pelo que o Pascal é tratado como *case-insensitive* nas suas palavras-chave (`BEGIN`, `Begin` e `begin` são equivalentes), tal como na linguagem original.

3.2 Comentários

São suportados os três formatos de comentário do Pascal, todos tratados pela mesma função `t_COMMENT`, através de uma única expressão regular com alternância:

- `{ comentário }`
- `(* comentário *)` (pode ocupar várias linhas)
- `// comentário` (até ao fim da linha)

O contador de linhas (`t.lexer.lineno`) é incrementado dentro desta função sempre que o comentário reconhecido contém mudanças de linha, para que os números de linha reportados em erros posteriores continuem corretos mesmo depois de um comentário multi-linhas.

3.3 Strings e números

- **Strings:** delimitadas por plicas simples (`'...'`), com suporte para `''` dentro de uma string como forma de representar literalmente uma plica, com implementação da regra padrão do Pascal (`t_STRING_LITERAL`), tratada com `t.value[1:-1].replace("'", "'')`.
- **Números inteiros:** reconhecidos por `\d+` e convertidos diretamente para `int` no momento do reconhecimento do token, para que o parser e as fases seguintes já recebam o valor numérico e não a sua representação em texto.

3.4 Erros léxicos

Um carácter inesperado é reportado com o próprio carácter e o número da linha onde ocorreu:

```
def t_error(t):
    print(f"Carácter inesperado '{t.value[0]}' na linha {t.lineno}.")
    t.lexer.skip(1)
```

O lexer **não aborta** ao encontrar um carácter inválido. Apenas ignora esse carácter e continua a tentar tokenizar o resto do ficheiro. Diferente do comportamento das fases seguintes, que abortam o programa assim que detetam o primeiro erro. Não abortar nesta fase, permite, por exemplo, reportar vários caracteres inválidos numa só execução, em vez de apenas o primeiro.

4. Análise Sintática

A análise sintática está implementada em `parser.py`, usando `ply.yacc`, e produz uma **AST** (não uma árvore de derivação completa), recorrendo às classes definidas em `ast_nodes.py`. Cada regra da gramática tem uma função `p_regra` correspondente, cujo corpo constrói o nó da AST a partir dos valores já processados pelas subregras (`p[1]`, `p[2]`, ...).

4.1 Estrutura da gramática

Em forma simplificada, a gramática segue esta estrutura:

```
program          : PROGRAM ID SEMI block DOT

block            : subprogram_decls declarations compound_statement

declarations     : VAR var_declarations_list
                  | empty

subprogram_decls : subprogram_decls subprogram_decl
                  | empty

subprogram_decl  : FUNCTION ID LPAREN params RPAREN COLON type_spec SEMI block SEMI

compound_statement : BEGIN statement_list END

statement        : compound_statement
                  | assign_statement
                  | if_statement
                  | while_statement
                  | for_statement
                  | procedure_call_statement
                  | empty
```

Um `program` é composto por um `block`, e este mesmo não-terminal `block` é reutilizado tanto no programa principal como no corpo de cada função, fazendo com que cada função tenha o seu próprio conjunto de variáveis/declarações locais, sem duplicar regras da gramática. De ter em conta, o `subprogram_decls` aparece **antes** de `declarations` na definição de `block`, porque no Exemplo 5 do enunciado, a função `BinToInt` é declarada antes do `var` do programa principal.

Listas (lista de identificadores, lista de instruções, lista de parâmetros) são construídas por recursão à esquerda, o padrão mais eficiente num parser LALR como o gerado pelo `ply.yacc`:

```
id_list : id_list COMMA ID
        | ID
```

O `empty` (que reduz para nada) é utilizado na gramática para tornar o `var` opcional (`declarations`), para tornar a lista de subprogramas opcional (`subprogram_decls`), e para permitir um `;` a mais antes de um `end` (produzindo um `NoOp` na AST, em vez de um erro de sintaxe).

4.2 Precedência de operadores

A precedência e associação dos operadores está centralizada numa única tabela, do menos para o mais prioritário:

```
precedence = (
    ('nonassoc', 'IFX'),
    ('nonassoc', 'ELSE'),
    ('left', 'OR'),
    ('left', 'AND'),
    ('nonassoc', 'EQ', 'NE', 'LT', 'GT', 'LE', 'GE'),
    ('left', 'PLUS', 'MINUS'),
    ('left', 'TIMES', 'DIV', 'MOD'),
)
```

Isto garante, por exemplo, que `2 + 3 * 4` é interpretado corretamente (`2 + (3 * 4)`) sem necessidade de multiplicar regras da gramática por cada nível de precedência. Uma única regra `expression : expression op expression` cobre todos os operadores binários, e é o `ply.yacc` que decide, a partir desta tabela, a ordem correta de redução.

Os operadores de comparação (`=`, `<>`, `<`, `>`, `<=`, `>=`) são marcados como `nonassoc`, o que impede escrever `a < b < c`, uma vez que o Pascal Standard não permite encadear comparações sem parênteses.

4.3 Resolução do dangling-else

Na análise sintática, um dos pontos mais cruciais de resolver é a ambiguidade clássica do `if` sem `else` seguido de um `else`:

```
if a then if b then x else y
```

Sem qualquer tratamento, um parser LALR não sabe se o `else` pertence ao `if b` ou ao `if a`. A abordagem mais comum em livros-texto resolve isto reescrevendo a gramática com dois não-terminais separados (instrução "completa" e "incompleta"). Neste projeto optou-se por uma abordagem mais compacta: atribuir precedências artificiais aos próprios `if`:

```
def p_if_statement(p):
    """if_statement : IF expression THEN statement %prec IFX
                      | IF expression THEN statement ELSE statement """
```

O `%prec IFX` na primeira alternativa (sem `else`) atribui-lhe a precedência do token fictício `IFX`, que na tabela de precedência está definida como mais baixa do que `ELSE`. Quando o parser chega a um ponto em que pode tanto fechar o `if` mais interno (reduce) como continuar a ler um `else` que apareça a seguir (shift), a precedência mais baixa de `IFX` faz o parser preferir sempre o shift. Ou seja, continuar a ler o `else` e associá-lo ao `if` mais próximo, que é o comportamento correto e esperado em Pascal.

4.4 Tratamento de erros sintáticos

A função `p_error` é chamada automaticamente pelo `ply.yacc` sempre que ocorre um erro sintático, imprimindo o token e a linha onde ocorreu (ou uma mensagem de fim de ficheiro inesperado, se o erro ocorrer no fim do programa):

```
def p_error(p):
    if p:
        print(f"Erro de sintaxe '{p.value}' na linha {p.lineno}")
    else:
        print("Erro de sintaxe: fim de ficheiro inesperado")
```

O `yacc.yacc()` trata de devolver `None` da chamada a `parser.parse(...)` nestes casos, sem lançar uma exceção Python, e `main.py` termina o programa de forma controlada (`sys.exit(1)`) assim que deteta que a AST devolvida é `None`.

5. Análise Semântica

A análise semântica está implementada em `semantic.py`, e é responsável por validar tipos de dados, declarações de variáveis e a coerência geral do programa, percorrendo a AST já construída pelo parser.

5.1 Tabela de símbolos e âmbitos

A tabela de símbolos (`SymbolTable`) é implementada como uma pilha de dicionários: o topo da pilha representa o âmbito mais interno (o corpo de uma função), e a base representa o âmbito global.

```
class SymbolTable:
    def __init__(self):
        self._scopes = [{}]          # começa só com o âmbito global

    def push_scope(self):            # ao entrar numa função
        self._scopes.append({})

    def pop_scope(self):             # ao sair de uma função
        self._scopes.pop()

    def lookup(self, name):
        for scope in reversed(self._scopes):
            if name in scope:
                return scope[name]
        raise SemanticError(...)
```

Ao entrar no corpo de uma função é empilhado um novo âmbito vazio; ao sair, esse âmbito é removido (num bloco `try/finally`, para garantir que é sempre removido mesmo que a análise do corpo lance um erro a meio). A procura de uma variável (`lookup`) percorre a pilha do topo para a base, o que permite que o corpo de uma função aceda a variáveis globais, mas impede que as suas variáveis locais "vazem" para fora dessa função depois de esta terminar.

O **próprio nome da função** é declarado como uma variável do seu tipo de retorno, dentro do seu âmbito, antes de o corpo ser analisado:

```
self.symbols.declare(subprogram.name, signature.return_type)
```

Isto permite tratar `NomeDaFuncao := valor` (a forma Pascal de definir o valor de retorno de uma função) como uma atribuição normal, através do mesmo caminho usado para qualquer outra variável, sem necessidade de um caso especial na análise de instruções `Assign`.

5.2 Verificação de tipos

A verificação de tipos é sempre **exata** (por exemplo, não é permitido atribuir um `integer` a uma variável `string`, nem comparar um `integer` com um `boolean`). Esta decisão simplifica consideravelmente a análise, e é coerente com o Pascal Standard, que também não realiza este tipo de relações entre tipos incompatíveis.

As regras de tipo por categoria de operador são:

Operadores	Operandos exigidos	Resultado
'+', '-', '*', 'div', 'mod'	integer, integer	integer
'and', 'or'	boolean, boolean	boolean
'=', '<>'	mesmo tipo (qualquer escalar)	boolean
'<', '<=', '>', '>='	integer, integer	boolean

Um caso particular é o tratamento de **strings como "arrays de carácter"**: como a AST não distingue um tipo `char` à parte, indexar uma string (`bin[i]`) reutiliza o mesmo mecanismo de verificação usado para indexar um array (`_infer_array_access`), devolvendo sempre `string` como resultado (representando um único carácter).

5.3 Funções

As assinaturas de todas as funções de um bloco são registadas (número e tipo dos parâmetros, e o tipo de retorno) **antes** de se analisar o corpo de qualquer uma delas. Isto é feito em duas passagens separadas (`_register_function_signature` e depois `_visit_subprogram`).

Cada chamada de função é validada tendo em conta o número de argumentos e o tipo de cada um, comparando com a assinatura registada:

```
if len(call.args) != len(signature.param_types):
    raise SemanticError(
        f"Função '{call.name}' espera {len(signature.param_types)} "
        f"argumentos, mas recebeu {len(call.args)}")
```

A função embutida `length` (usada no Exemplo 5 para calcular o comprimento de uma string) é tratada como um caso especial, antes de se consultar a tabela de funções definidas pelo utilizador.

5.4 Exemplo de deteção de erro

Um dos casos de teste (`erro_semantico_tipos_incompativeis.pas`) tenta atribuir uma string a uma variável inteira. O analisador semântico deteta o erro em `_visit_statement`, no ramo de `Assign`, ao comparar o tipo inferido do lado esquerdo (`integer`) com o tipo inferido do lado direito (`string`):

```
FALHOU: Tipos incompatíveis em atribuicao: integer vs string
```

O programa termina imediatamente após este erro, sem avançar para a geração de código. Não faria qualquer sentido avançar para a geração de código de algo errado.

6. Geração de Código

A geração de código está implementada em `codegen.py` e traduz a AST (já validada) diretamente em instruções da EWVM, sem passar por uma representação intermédia.

6.1 Variáveis globais

Cada variável global recebe um índice sequencial, atribuído pela ordem em que é declarada (`_collect_globals`), e é acedida através de `PUSHG / STOREG`. No arranque do programa, `PUSHN n` reserva de uma só vez o espaço para todas as variáveis globais.

6.2 Frames de função

Dentro de uma função, é usado um **frame** com três zonas de offsets relativos ao apontador de frame (`fp`):

Zona	Offset	Instrução
Valor de retorno	mais negativo (<code>-(n+1)</code>)	<code>PUSHL / STOREL</code>
Parâmetros	negativos (<code>fp[-1]</code> , <code>fp[-2]</code> , ...)	<code>PUSHL / STOREL</code>
Variáveis locais	não-negativos (<code>fp[0]</code> , <code>fp[1]</code> , ...)	<code>PUSHL / STOREL</code>

O último parâmetro fica em `fp[-1]`, o primeiro em `fp[-n]` — a ordem inversa da declaração, que corresponde à ordem em que os argumentos são empilhados antes da chamada. Este é o esquema clássico de frame para uma máquina virtual baseada em pilha, como é o caso da EWVM.

6.3 Arrays

Cada variável do tipo `array` é alocada **uma única vez**, na heap, logo no arranque do programa (`ALLOCN`), guardando o endereço devolvido no slot correspondente a essa variável:

```
PUSHI 5      ; tamanho do array
ALLOCN
STOREG 0     ; guarda o endereço no slot da variável
```

Todos os acessos posteriores (leitura ou escrita) reutilizam esse endereço, combinando-o com o deslocamento do índice (`LOADN` / `STOREN`), em vez de alocar memória a cada acesso. Foi uma decisão tomada que melhora a eficiência, porque evita fragmentação de heap desnecessária ao longo da execução do programa.

6.4 Chamada de funções

Uma chamada de função usada como expressão (por exemplo, `valor := BinToInt(bin)`) segue a convenção:

```
PUSHI 0      ; reserva o slot do valor de retorno
<argumentos> ; cada argumento é empilhado, pela ordem declarada
PUSHA nome   ; endereço (rótulo) da função
CALL
POP n        ; remove os n argumentos, deixando só o valor de retorno
```

6.5 Estruturas de controlo

`if`, `while` e `for` são traduzidos para saltos condicionais (`JZ`) e incondicionais (`JUMP`), com rótulos únicos gerados por um contador interno (`new_label`), o que evita colisões de nomes quando existem várias instruções de controlo de fluxo no mesmo programa. Por exemplo, um `while` é traduzido segundo o padrão:

```
whileN:
    <condição>
    JZ endwhileN
    <corpo>
    JUMP whileN
endwhileN:
```

e um `for ... to ...` inclui, adicionalmente, a inicialização da variável de controlo e o seu incremento a cada iteração, com a condição de paragem recalculada em cada volta do ciclo (`variável <= limite`).

6.6 Exemplo completo: do código-fonte ao `.vm`

Para ilustrar o pipeline completo, considere-se o Exemplo 2 do enunciado (cálculo do fatorial):

```
program Fatorial;
var
    n, i, fat: integer;
begin
    writeln('Introduza um numero inteiro positivo:');
    readln(n);
    fat := 1;
    for i := 1 to n do
        fat := fat * i;
    writeln('Fatorial de ', n, ': ', fat);
end.
```

A análise léxica reconhece 53 tokens, entre palavras reservadas (`PROGRAM`, `VAR`, `BEGIN`, `FOR`, `TO`, `DO`, `END`), identificadores (`n`, `i`, `fat`), literais (`NUM_INT 1`, `STRING_LITERAL`) e pontuação. A análise sintática e semântica não reportam qualquer erro (as três variáveis são `integer`, e todos os operadores são usados com operandos do tipo correto). A geração de código produz o seguinte `.vm`:

```
START
PUSHN 3
JUMP main1
main1:
PUSHS "Introduza um numero inteiro positivo:"
WRITES
Writeln
READ
ATOI
STOREG 0
PUSHI 1
STOREG 2
PUSHI 1
STOREG 1
for2:
PUSHG 1
PUSHG 0
INFEQ
JZ endfor3
PUSHG 2
PUSHG 1
MUL
STOREG 2
PUSHG 1
PUSHI 1
ADD
STOREG 1
JUMP for2
endfor3:
PUSHS "Fatorial de "
WRITES
PUSHG 0
WRITEI
PUSHS ": "
WRITES
PUSHG 2
WRITEI
Writeln
STOP
```

Note-se: `PUSHN 3` reserva espaço para as três variáveis globais (`n` no índice 0, `i` no índice 1, `fat` no índice 2, pela ordem em que aparecem na lista `n`, `i`, `fat`); o `readln(n)` lê uma string do teclado (`READ`) e converte-a para inteiro (`ATOI`) antes de a guardar; o ciclo `for` gerado entre os rótulos `for2` e `endfor3` testa `i <= n` (`INFEQ`) antes de cada iteração, e incrementa `i` (`ADD`) no final de cada volta; e o `writeln` final intercala texto (`WRITES`) com valores inteiros (`WRITEI`), terminando sempre com um `Writeln` que emite a mudança de linha.

6.7 Instruções EWVM utilizadas

A tabela seguinte resume as instruções da EWVM emitidas pelo gerador de código, agrupadas por categoria:

Categoria	Instruções
Controlo do programa	START , STOP , JUMP , JZ , rótulos (nome :)
Pilha / memória	PUSHN , PUSHI , PUSHS , PUSHG , PUSHL , PUSHA , STOREG , STOREL , POP
Heap (arrays)	ALLOCN , LOADN , STOREN
Aritmética	ADD , SUB , MUL , DIV , MOD
Lógicos / comparação	AND , OR , NOT , EQUAL , INF , INFEQ , SUP , SUPEQ
Chamadas de função	CALL , RETURN
Entrada / saída	READ , WRITEI , WRITES , Writeln

7. Testes

Os testes do projeto assentam em cinco programas de exemplo, correspondentes aos exemplos dados no próprio enunciado (`testes/exemplos_corretos/`), cada um acompanhado de um ficheiro de input e do respetivo output esperado:

Exemplo	Conceitos exercitados
HelloWorld	Programa mínimo, <code>writeln</code>
Fatorial	<code>for</code> , leitura de inteiros
NúmeroPrimo	<code>while</code> , operadores lógicos e <code>mod</code>
SomaArray	Arrays, <code>for</code> com leitura indexada
BinarioParaInteiro	Funções, <code>string</code> , <code>length</code> , <code>for downto</code>

Estes cinco exemplos cobrem, no seu conjunto, todas as construções obrigatórias do enunciado: declaração de variáveis, expressões aritméticas e lógicas, as três estruturas de controlo (`if`, `while`, `for`), arrays, e subprogramas com parâmetros e valor de retorno.

Adicionalmente, existem cinco casos de teste de erro (`testes/exemplos_erros/`), um por cada tipo de erro que o compilador deve detetar:

Ficheiro	Tipo de erro
<code>erro_lexico_caracter_invalido.pas</code>	Léxico — carácter inválido
<code>erro_sintatico_falta_semi.pas</code>	Sintático — <code>;</code> em falta
<code>erro_semantico_var_nao_declarada.pas</code>	Semântico — variável não declarada
<code>erro_semantico_tipos_incompativeis.pas</code>	Semântico — tipos incompatíveis numa atribuição
<code>erro_semantico_funcao_argumentos_errados.pas</code>	Semântico — número/tipo de argumentos incorreto

Cada um destes casos foi confirmado manualmente como sendo detetado pela fase correspondente do compilador, com uma mensagem de erro identificando a causa. Fica como trabalho a formalizar a comparação automática dessas mensagens face a um valor esperado (ver secção seguinte).

8. Limitações e Trabalho Futuro

- **procedure não suportada**: a palavra reservada existe no lexer, mas a gramática só produz declarações de `function`. Um procedimento pode, no entanto, ser simulado com uma função cujo valor de retorno não é usado.
- **Arrays de uma só dimensão**: a gramática e o gerador de código só suportam `array[first..last]` of `tipo`, não matrizes multidimensionais.
- **Sem otimização de código**: as etapas opcionais de otimização (remoção de redundâncias, otimizações locais/globais) não foram implementadas, dado tratar-se de um extra do enunciado.
- **Sem representação intermédia separada**: optou-se por gerar diretamente instruções EWVM, pelo que a etapa opcional de "geração de código final" a partir de uma representação intermédia não se aplica a este projeto.
- **Testes automatizados por reconstruir**: os casos de erro e os exemplos corretos são validados manualmente; não existe ainda um script (ex.: `pytest`) que corra o compilador sobre cada exemplo, execute o `.vm` resultante na EWVM, e compare automaticamente com o output esperado.
- **Trabalho futuro natural**: suporte a `procedure`, arrays multidimensionais, e a reconstrução da suite de testes automatizados seriam as extensões mais diretas a partir do estado atual do projeto.

9. Conclusão

O compilador implementado cobre de forma completa as quatro etapas obrigatórias definidas no enunciado (análise léxica, análise sintática, análise semântica e geração de código), sendo capaz de processar corretamente os cinco exemplos de referência fornecidos, incluindo declaração de variáveis, expressões aritméticas e lógicas, estruturas de controlo de fluxo (`if`, `while`, `for`), arrays e funções com parâmetros e valor de retorno. As decisões de implementação mais relevantes são de facto, a resolução do dangling-else por precedência, o esquema de frame para funções, e a alocação única de arrays na heap.